Blockchain : Experiment - 8

Aim:

Implement the embedding wallet (Metamask) and transaction using Solidity.

Theory:

The Problem in a Pharmaceutical Supply Chain

A medicine passes through several independent organisations before it reaches a patient: the manufacturer produces it, a distributor moves it in bulk, a retailer or pharmacy stocks it, and finally a customer buys it. Each party keeps its own private records, so no single party can see the complete history of a particular batch. This creates three well-known weaknesses. Counterfeit medicines can be injected at any handover because there is no shared proof of origin; expired stock can be sold because the expiry check depends on whoever happens to be holding the box; and when something does go wrong, a recall is slow because the trail has to be reconstructed from several disconnected databases.

Why Blockchain Suits This Problem

A blockchain replaces those separate ledgers with one shared, append-only record that every participant can read but none can rewrite. Once a handover is written into a block it is secured by the hash chain, so altering it would require rewriting every block that follows and out-computing the rest of the network. For a supply chain this gives three properties that matter: traceability, because the full custody history of a drug ID is on the chain; immutability, because a recorded handover cannot be quietly edited later; and transparency, because every participant sees the same state at the same time rather than trusting a copy sent by someone else.

Smart Contracts and Solidity

A smart contract is a program stored on the blockchain that executes exactly as written whenever it is invoked. It is not a legal document but ordinary code, and the network itself enforces it — there is no administrator who can be persuaded to make an exception. Solidity is the statically-typed language used to write contracts for the Ethereum Virtual Machine. Deploying a contract is itself a transaction: the compiled bytecode is sent to the network, the contract is given a permanent address, and from then on anyone can call its public functions.

Design of the DrugSupplyChain Contract

The contract models the journey of a drug as a small state machine and refuses any move that does not follow the intended order.

- enum Stage - the four states a drug can be in: Manufactured, WithDistributor, WithRetailer and Sold. Storing the stage as an enum means an invalid state cannot be written at all.
- struct Drug - the record held for each drug: its ID, name and expiry date, the four addresses (manufacturer, distributor, retailer, customer), the current stage and the timestamp of the last update.
- mapping(uint256 => Drug) - the on-chain store, keyed by drug ID. Declaring it public makes Solidity generate a free getter, so any drug record can be read without a transaction.
- Events - DrugManufactured, SentToDistributor, SentToRetailer and SoldToCustomer are emitted at each handover. Events are written to the transaction log, which is far cheaper than storage and is what an external application listens to in order to track movement.
- require() checks - every function validates both who is calling and what state the drug is in before changing anything. If a check fails the whole transaction is reverted and the state is left untouched.

The access rules are what make the chain of custody meaningful. Only the manufacturer who created a drug may send it to a distributor, only that distributor may forward it to a retailer, and only that retailer may sell it to a customer. The final sale additionally checks block.timestamp < d.expiryDate, so an expired

drug cannot be sold even by the correct retailer. Because each function also requires the drug to be in the expected stage, a step cannot be skipped or repeated.

Test Networks and the Sepolia Testnet

Deploying to Ethereum mainnet costs real ether, so development is done on a test network. Sepolia is the Ethereum test network used here; it behaves like mainnet and has chain ID 11155111, but its ether has no monetary value and is obtained free from a faucet. This allows a contract to be deployed and exercised repeatedly at no cost before any real deployment is considered.

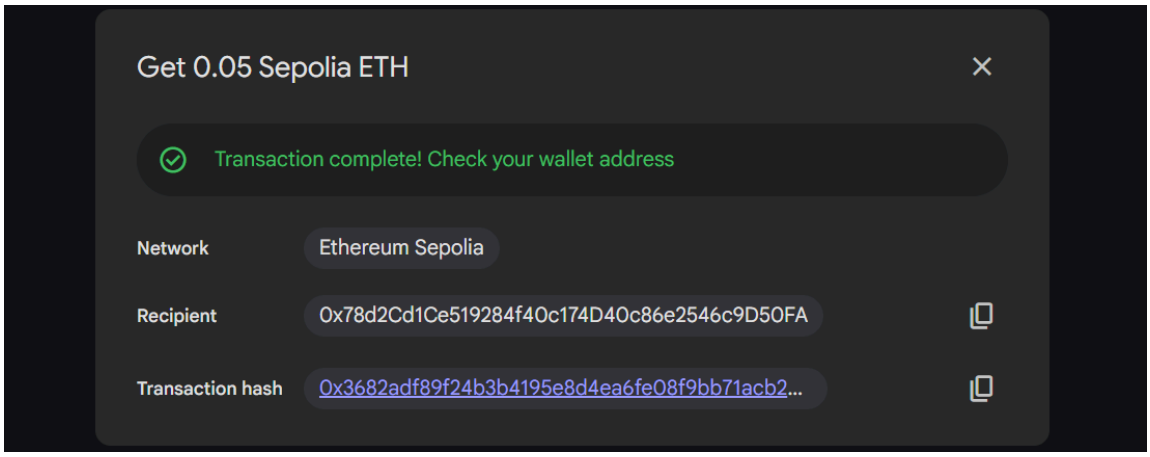
Code :

```
pragma solidity ^0.8.0;
contract DrugSupplyChain {
  enum Stage { Manufactured, WithDistributor,
WithRetailer, Sold }
  struct Drug {
    uint256 drugID;
    string name;
    uint256 expiryDate;
    address manufacturer;
    address distributor;
    address retailer;
    address customer;
    Stage stage;
    uint256 lastUpdated;
  }
  mapping(uint256 => Drug) public drugs;
  event DrugManufactured(uint256 drugID,
string name, address manufacturer);
  event SentToDistributor(uint256 drugID,
address distributor);
  event SentToRetailer(uint256 drugID, address
retailer);
  event SoldToCustomer(uint256 drugID,
address customer);
  function manufactureDrug(uint256 _drugID,
string memory _name, uint256 _expiryDate)
public {
    require(_drugID != 0, "Invalid drug ID");
    require(drugs[_drugID].drugID == 0, "Drug
already exists");
    drugs[_drugID] = Drug({
      drugID: _drugID,
      name: _name,
      expiryDate: _expiryDate,
      manufacturer: msg.sender,
      distributor: address(0),
      retailer: address(0),
      customer: address(0),
      stage: Stage.Manufactured,
      lastUpdated: block.timestamp
    });
    emit DrugManufactured(_drugID, _name,
msg.sender);
  }
  function sendToDistributor(uint256 _drugID,
address _distributor) public {
    Drug storage d = drugs[_drugID];
    require(d.drugID != 0, "Drug does not
exist");
    require(msg.sender == d.manufacturer,
"Only manufacturer allowed");
    require(d.stage == Stage.Manufactured,
"Wrong stage");
    d.distributor = _distributor;
    d.stage = Stage.WithDistributor;
    d.lastUpdated = block.timestamp;
    emit SentToDistributor(_drugID,
_distributor);
  }
  function sendToRetailer(uint256 _drugID,
address _retailer) public {
    Drug storage d = drugs[_drugID];
    require(d.drugID != 0, "Drug does not
exist");
    require(msg.sender == d.distributor, "Only
distributor allowed");
    require(d.stage == Stage.WithDistributor,
"Wrong stage");
    d.retailer = _retailer;
    d.stage = Stage.WithRetailer;
    d.lastUpdated = block.timestamp;
    emit SentToRetailer(_drugID, _retailer);
  }
  function sellToCustomer(uint256 _drugID,
address _customer) public {
    Drug storage d = drugs[_drugID];
```

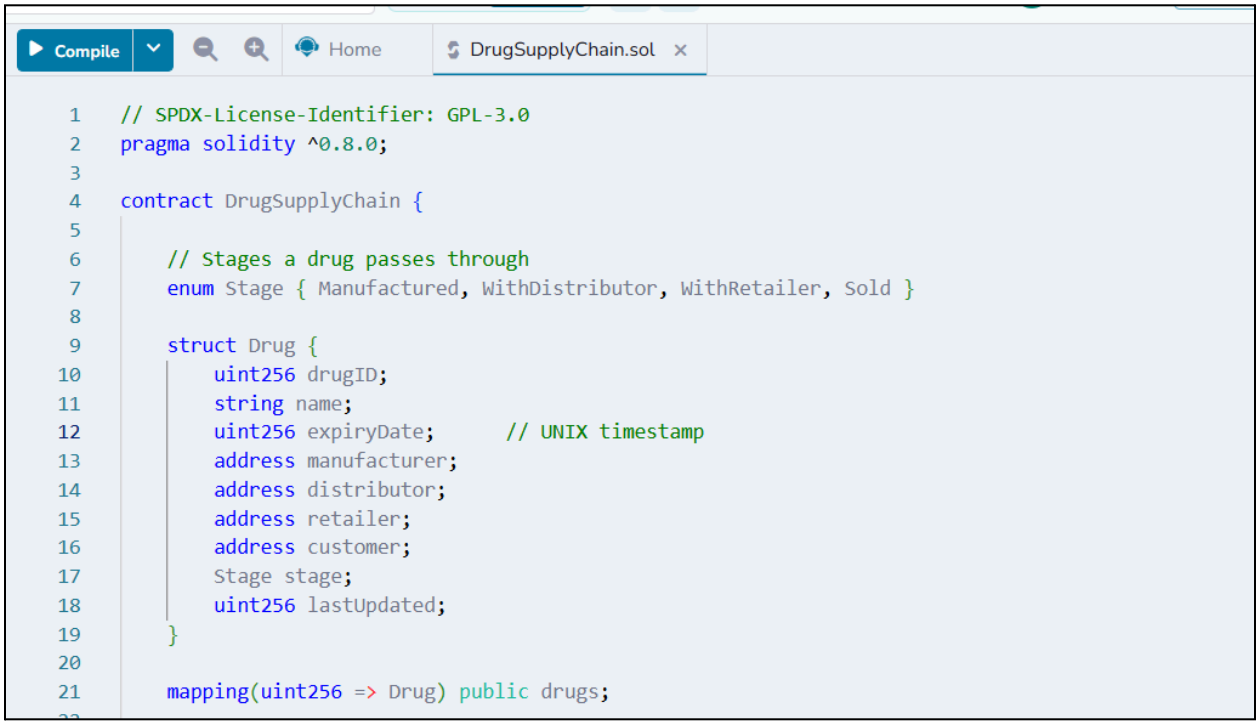
```
        require(d.drugID != 0, "Drug does not exist");
        require(msg.sender == d.retailer, "Only retailer allowed");
        require(d.stage == Stage.WithRetailer, "Wrong stage");
        require(block.timestamp < d.expiryDate, "Drug expired");
        d.customer = _customer;
        d.stage = Stage.Sold;
        d.lastUpdated = block.timestamp;
        emit SoldToCustomer(_drugID, _customer);
    }
    function getDrug(uint256 _drugID) public view returns (Drug memory) {
        require(drugs[_drugID].drugID != 0, "Drug does not exist");
        return drugs[_drugID];
    }
}
```

Output:

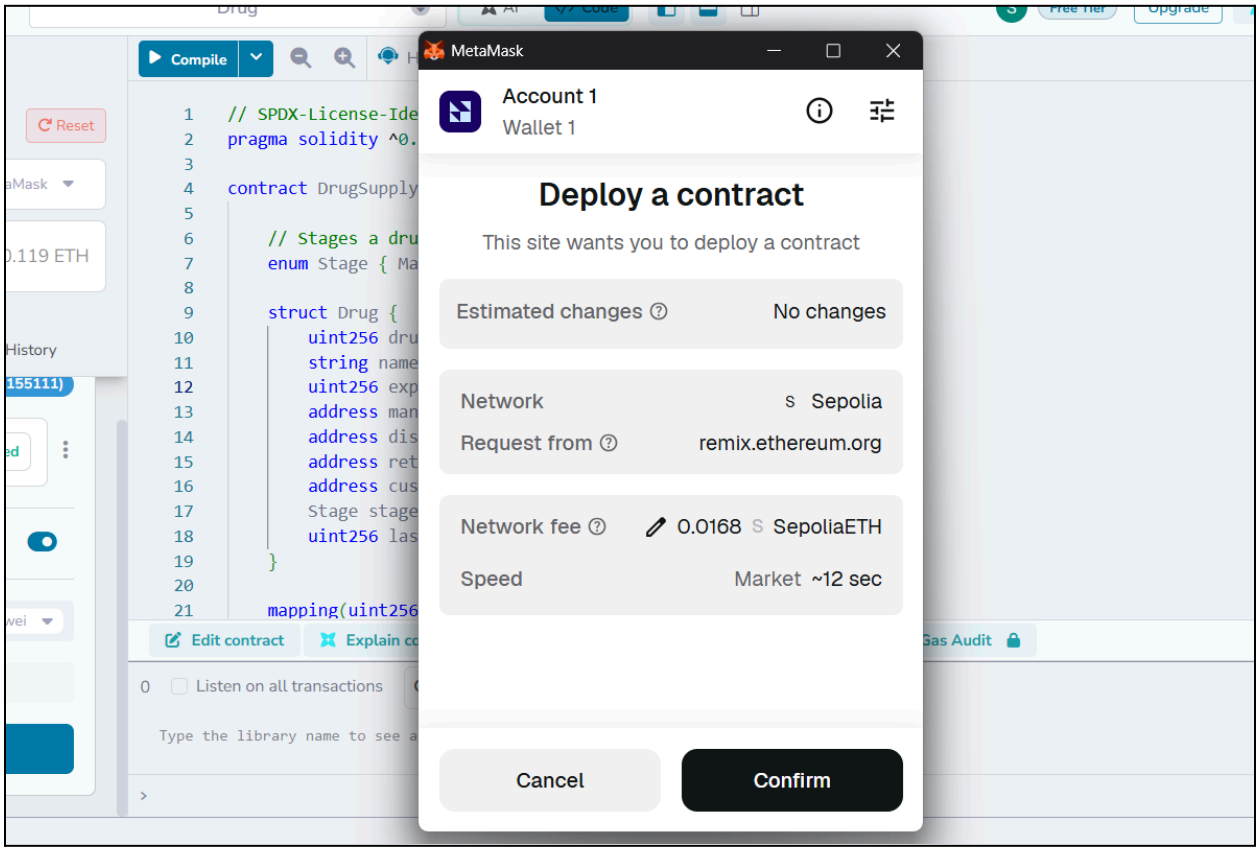
Get Sepolia Testnet Eth:



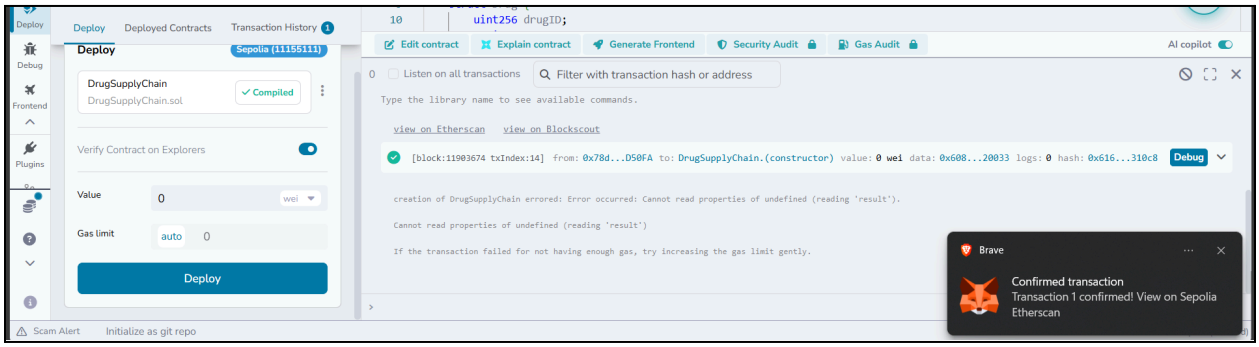
Create Smart Contract:



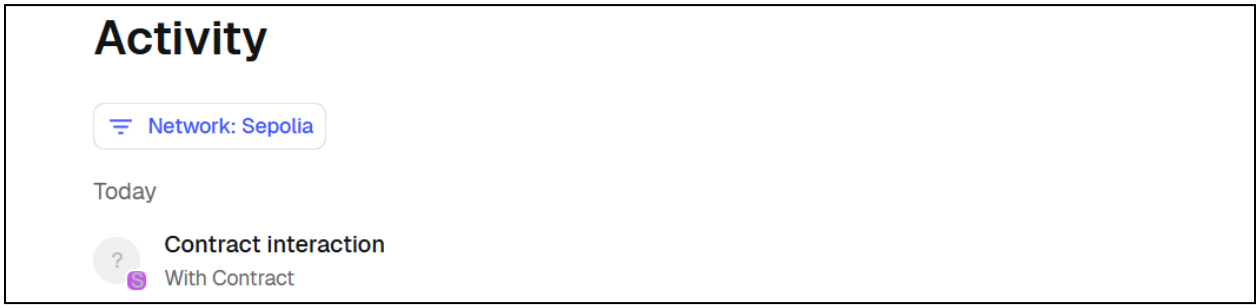
Deploy Smart Contract using Sepoli Testnet and Metamask:

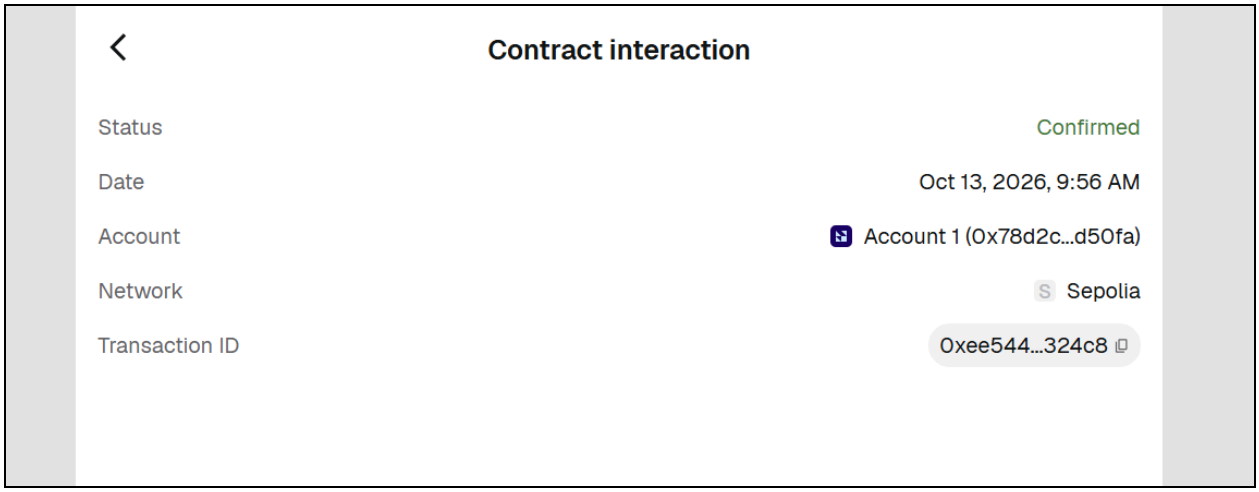


Deployed Smart Contract:

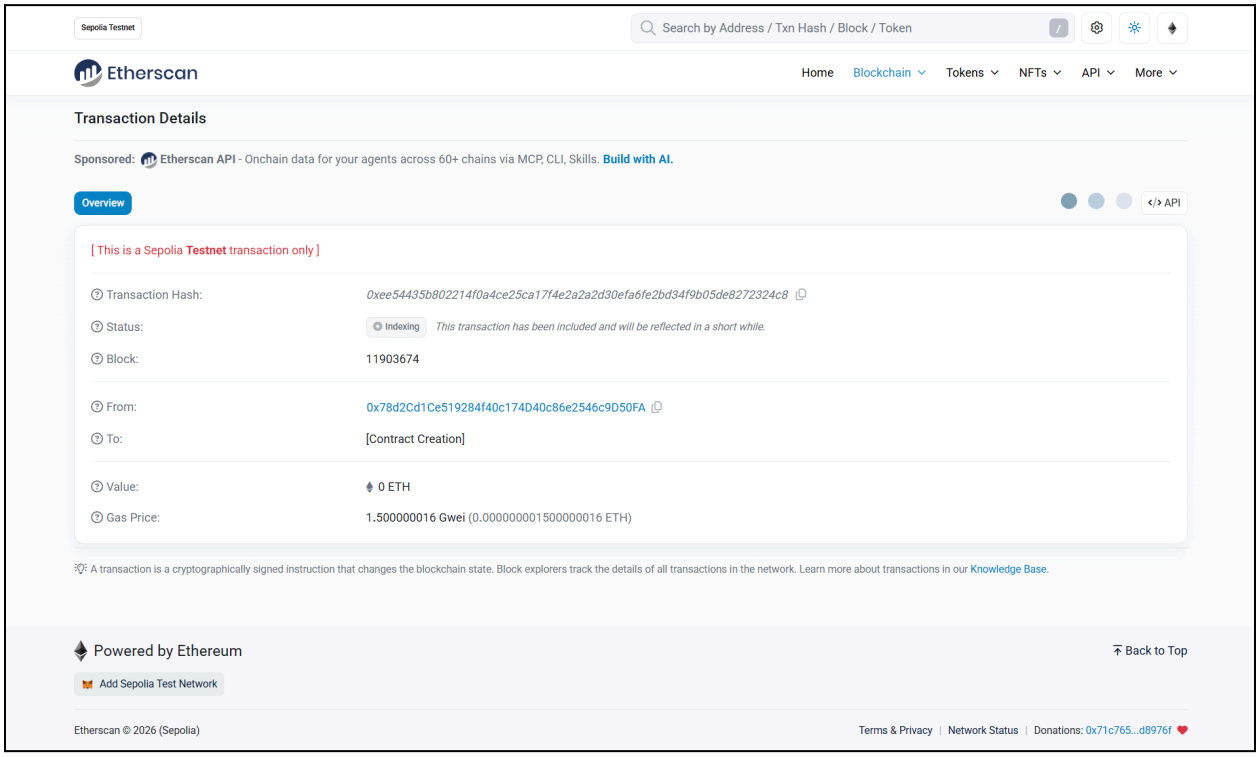


Smart Contract Transactions:





Check the transaction on Etherscan:



Observation:

1. The faucet credited 0.05 Sepolia ETH to the account, which was enough to cover the deployment; the Remix sidebar showed a balance of 0.119 ETH at the time of deploying.
2. The contract compiled on a Solidity 0.8.x compiler with no errors and no warnings, and produced six public functions and four events.
3. MetaMask reported the deployment fee as 0.0168 SepoliaETH, and Etherscan later showed the gas price as 1.500000016 Gwei — a deployment is costly because the whole bytecode is stored on a chain.
4. The transaction value was 0 ETH and the To field read [Contract Creation] rather than an address, which is how Ethereum represents a deployment as opposed to a transfer.

5. The Remix terminal printed "creation of DrugSupplyChain errored: Cannot read properties of undefined (reading ‘result’)", yet the MetaMask notification reported the transaction as confirmed and Etherscan showed it mined in block 11903674. The error was therefore raised by the Remix front end while reading the receipt, not by the network — the deployment itself succeeded.
6. Etherscan briefly showed the status as "Indexing" rather than Success, which reflects the explorer still processing the block and not a problem with the transaction.
7. Because the stage is stored as an enum and every function checks both the caller and the current stage, the handover order is enforced by the contract itself: a drug cannot be sent to a retailer before it has reached a distributor, and no step can be repeated.
8. The expiry check in sellToCustomer compares block.timestamp with the stored expiryDate, so the network clock decides whether a sale is allowed rather than any participant.

Conclusion:

A smart contract for a pharmaceutical drug supply chain was designed in Solidity and deployed successfully on the Ethereum Sepolia test network. The contract models the journey of a drug as a four-stage state machine and uses require() checks so that only the current holder may pass a drug to the next party, no stage can be skipped, and an expired drug cannot be sold. Test ether was obtained from a faucet, the contract was compiled and deployed from Remix through MetaMask, and the resulting transaction was traced in the wallet and verified on Etherscan in block 11903674. The experiment therefore demonstrated the complete workflow of writing, deploying and verifying a smart contract on a public test network, and showed how blockchain brings traceability, immutability and transparency to a supply chain that would otherwise depend on several disconnected private records.